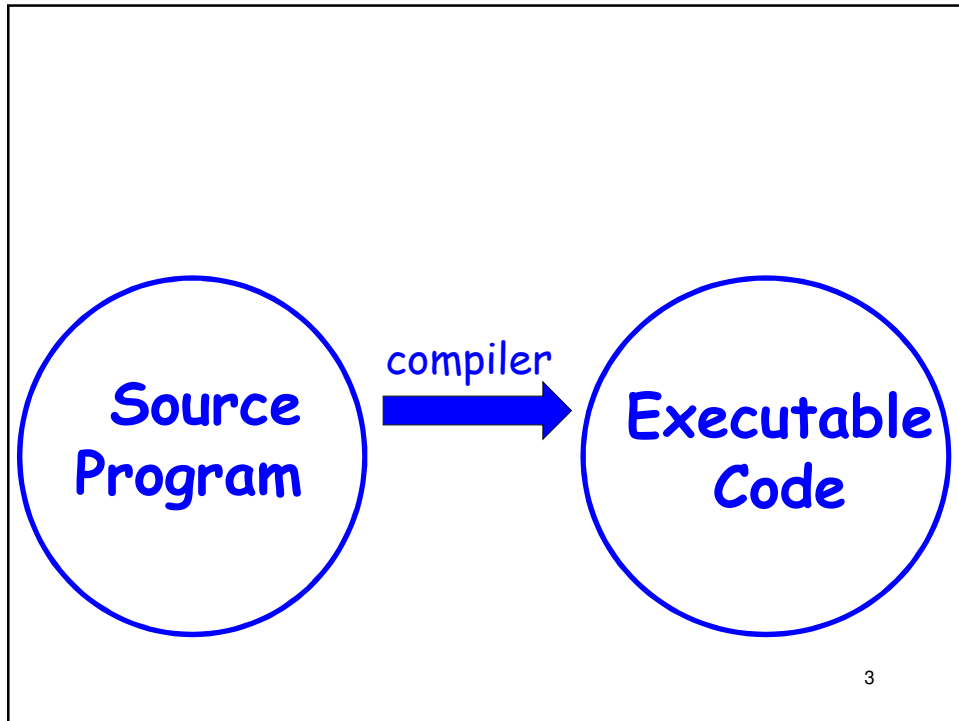


FORMAL LANGUAGES AND COMPILERS

Paola Quaglia

Goals & Expected Learning

- Concepts, techniques, and algorithms for the implementation of compilers
- Formal languages
- Recognition of formal languages



A long way to go

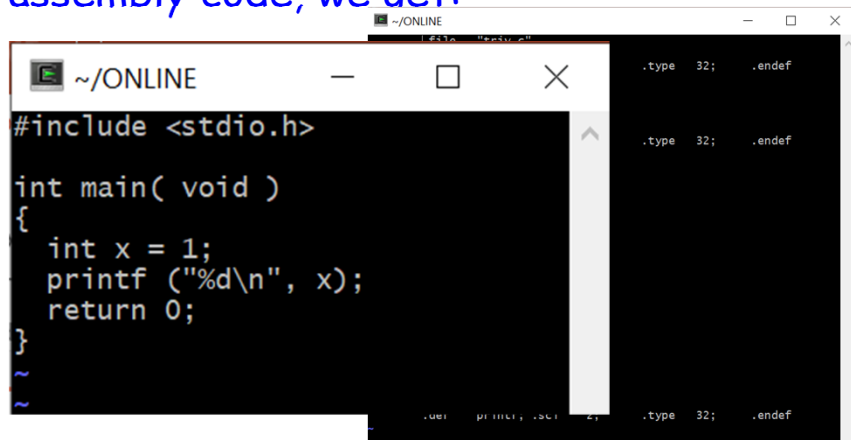
A long way to go

- E.g., the development of gcc has been going on for more than 25 years by now
- Some years ago it already consisted of 2++ million lines of code

5

A long way to go

E.g., from source program to assembly code, we get:



The screenshot shows a code editor window titled '~ / ONLINE'. The left pane displays C source code:

```
#include <stdio.h>

int main( void )
{
    int x = 1;
    printf ("%d\n", x);
    return 0;
}
```

The right pane displays the corresponding assembly code, showing instructions like `.type 32; .endef` and `.type 32; .endef`.

E.g.,
assembler

```
~ / ONLINE
.file "triv.c"
.text
.def __main; .sc1 2; .type 32; .endif
.LC0:
.ascii "%d\\12\\0"
.text
.globl main
.def main; .sc1 2; .type 32; .endif
.seh_proc main
pushq %rbp
.seh_pushreg %rbp
movq %rsp, %rbp
.seh_setframe %rbp, 0
subq $48, %rsp
.seh_stackalloc 48
.seh_endprologue
call __main
movl $1, -4(%rbp)
movl -4(%rbp), %eax
movl %eax, %edx
leaq .LC0(%rip), %rcx
call printf
movl $0, %eax
addq $48, %rsp
popq %rbp
ret
.seh_endproc
.ident "GCC: (GNU) 9.3.0"
.def printf; .sc1 2; .type 32; .endif

~ / ONLINE
#include <stdio.h>
int main( void )
{
    int x = 1;
    printf ("%d\\12\\0", x);
    return 0;
}
```

A long way to go

- And assembly code is not yet executable
- Still needed:
 - Conversion of textual instructions to binary instructions
 - Linking to include libraries

Various phases in pipe line

- What we do have:

- The program is written in a language which adheres to a specific grammar
- Some programs are syntactically legal, some are not
- The grammar lets us decide on this

9

Various phases in pipe line

```
pippo = 1;  
pluto = 2;
```

```
while = 1;  
pluto = 2;
```

In most languages a keyword,
cannot be used as an identifier

OK

NOK

10

The grammar

- States the form of an assignment
 - IDENTIFIER ASSIGN EXPRESSION SEMICOLON
- Cannot know "pippo", "pluto", nor the infinitely many strings we may use as variables

11

Lexical analysis

- Translates a stream of characters into a stream of **tokens**
- E.g. translates
 - pippo = 2*3;
 - Into
 - <ID,pippo> ASS <NUM,2> MUL
 - <NUM,3> SEMCOL
 - ID,ASS,NUM,MUL,SEMCOL are tokens

12

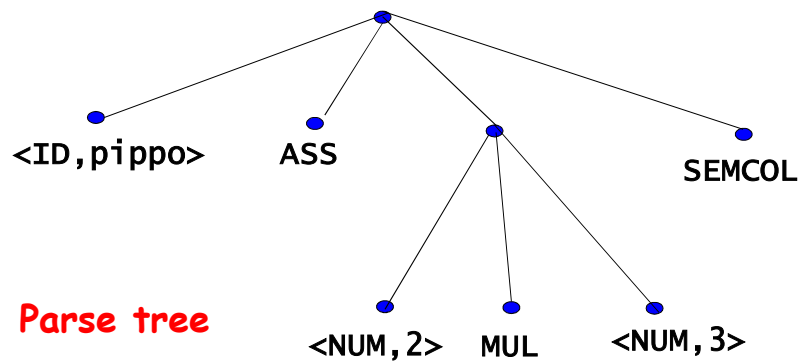
Syntax analysis

- Checks whether a stream of tokens like, e.g.,
 - **<ID,pippo> ASS <NUM,2> MUL <NUM,3> SEMCOL**
- Adheres to the given grammar
- If so, converts the stream of tokens into a **parse tree** (or better, into an **abstract syntax tree**)

13

Syntax analysis

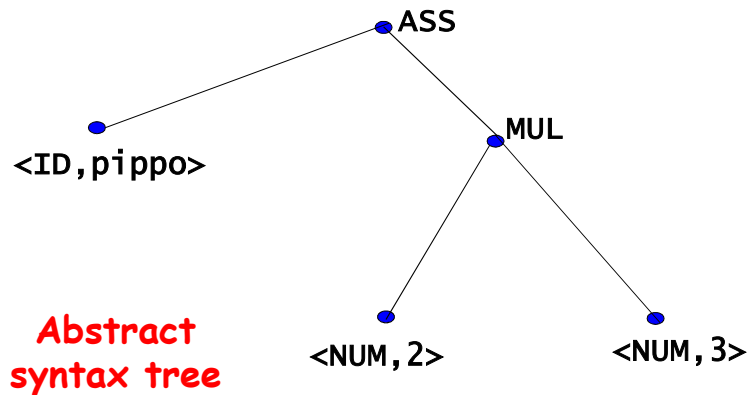
<ID,pippo> ASS <NUM,2> MUL <NUM,3> SEMCOL



14

Syntax analysis

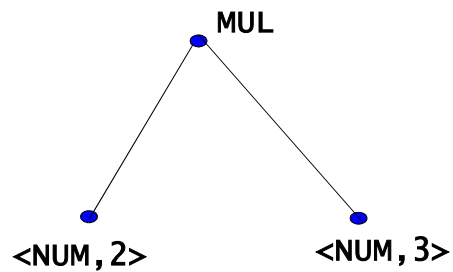
<ID,pippo> ASS <NUM,2> MUL <NUM,3> SEMCOL



15

Semantic analysis

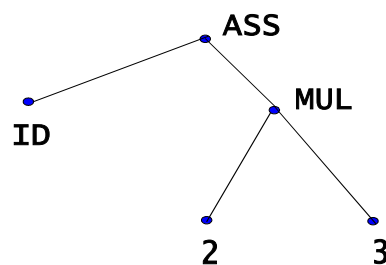
Are we actually multiplying two integers?



16

Intermediate code generation

- Converts a parse tree into a textual intermediate code



```
MOVE 2,R1
MULT 3,R1
MOVE R1,ID
```

17

Target code generation

- Translates intermediate code into target machine code (assembly code)

```
file "triv.c"
.text
.def __main; .sc1 2; .type 32; .endef
.section .rdata,"dr"
.LC0:
.ascl1 "%d12.0"
.text
.globl main
.def main; .sc1 2; .type 32; .endef
.seh_proc main
main:
pushq %rbp
.seh_pushreg %rbp
movq %rsp, %rbp
.seh_setframe %rbp, 0
subq $48, %rbp
.seh_stackalloc 48
.seh_endprologue
call __main
movl $1, -4(%rbp)
movl -4(%rbp), %eax
movl %eax, %edx
leaq 10(%rip), %rcx
call printf
movl $0, %eax
addq $48, %rbp
popq %rbp
ret
.seh_endproc
.ident "GCC: (GNU) 9.3.0"
.def printf; .sc1 2; .type 32; .endef
```

Plausible targets

- Machine code not necessarily the target of compilation
 - May be a virtual machine
 - May be another language

19

Front-end & Back-end

- Front-end
 - From lexical analysis all the way down to intermediate code generation
- Back-end:
 - All the rest
- Modularity:
 - N languages, N front-ends
 - K machines, K back-ends
 - $N \times K$ compilers, overall

20

That said

21

Goals & Expected Learning

- Pervasive structures (finite state automata, grammars, but also trees and graphs)
- Methodologies at the basis of other areas, too (e.g., parsing of natural languages, soundness of queries to databases)

22

In a Nutshell

- Regular languages and finite state automata (lexical analysis)
- Context-free languages and parsing (syntax analysis)
- Syntax driven definitions (semantic analysis & symbol tables)

23

In a Nutshell (ctd.)

- Intermediate code generation
- Machine code generation
- Optimization techniques

24

Main Textbook

- Compilers: Principles, Techniques, and Tools (2nd edition); Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman; Prentice Hall, 2006

25

Verification

- Written test +
- Oral test

26

Written Test

- Closed books & No electronics
- 5 questions
 - Questions on basics (general knowledge of the subject)
 - "Mechanical" questions (i.e. application of the algorithms studied in the course to a given structure)
 - "Show-off" questions (possibility to show a deeper understanding of the subject)

27

Written Test

- Compulsory subscription on esse3
 - In case of subscription & no-show the student undergoes a skip-session

28

Evaluation

- Final score: written and oral contribute 50% each
- *Lode* comes from 30/30 at the written test plus an excellent oral examination

29

Preliminaries: Generative Grammars

30

Generative Grammars Informally

- Fix a vocabulary
 - A set of symbols: **terminals** and **non-terminals**
 - **Terminals**, play the tokens of the output stream of lexical analysis
- E.g. take the vocabulary {S, a, b} where "a" and "b" are terminals

31

Generative Grammars

- One non-terminal symbol of the vocabulary is chosen as **start symbol**
- E.g., S in {S, a, b}

32

Generative Grammars

- Fix a set of **productions**
 - Rules for rewriting strings into strings
 - Constraint:
 - the string to be replaced must contain at least a non-terminal
- E.g. $\{S \rightarrow aSb, S \rightarrow ab\}$

33

Generative Grammars

- These are the ingredients of a **generative grammar**
- A **language** of words of terminals can be generated from the start symbol:
 - Apply the rewriting rules in any possible way, as many times as possible
 - Each rewriting is called a **derivation step**

34

Notation for the derivation relation

$\{S \rightarrow aSb, S \rightarrow ab\}$
 $S \Rightarrow ab$

- Is a one-step derivation from S
- "ab" is made up of terminals only
- Hence "ab" **belongs** to the language generated by the given grammar

35

$\{S \rightarrow aSb, S \rightarrow ab\}$

$S \Rightarrow aSb \Rightarrow aabb$

- Is a two-step derivation from S
- "aabb" is made up of terminals only
- Hence "aabb" **belongs** to the language generated by the given grammar

36

$\{S \rightarrow aSb, S \rightarrow ab\}$

$S \Rightarrow aSb \Rightarrow aaSbb$

- Is a two-step derivation of a string from S
- But "aaSbb" contains a non-terminal
- Hence "aaSbb" **does not belong** to the language generated by the given grammar

37

$\{S \rightarrow aSb, S \rightarrow ab\}$

- Which is the language generated by this grammar?
- $\{a^n b^n \mid n > 0\}$

38

Notation

- Capital letters for non-terminals

39

Convention

- Special character epsilon (ϵ) used to denote the empty word
- Length of ϵ is 0
 - $\epsilon \equiv \epsilon \epsilon$
 - $\epsilon \equiv b^0$ for every terminal b

40